

	A	B	C	D
1	Date	Project	Customer	Hours
2	01-Feb-15	ProjectA	Jackie Chan	10
3	03-Feb-15	ProjectA	Jackie Chan	8
4	04-Feb-15	ProjectA	Jackie Chan	8
5	05-Feb-15	ProjectB	Jet Li	8
6	06-Feb-15	ProjectB	Jet Li	8
7	07-Feb-15	ProjectA	Jackie Chan	8

Figure 3: Sample data

```

for each report file in a directory
read each line and update corresponding customer hours
summary
when done, print the summary
'''

# import glob object to read xlsx file names
import glob
# import xl reader module
import xlrd

# get date and datetime
from datetime import date, datetime, time
from xlrd import open_workbook, xldate_as_tuple

# note position of different data items in the summary file
DATECOL = 0
CUSTOMERCOL = 2
HOURSCOL = 3

# constant to store value to check for a week's data
ONeweek = 7
ONLYDATE = 3

# customer constants used
CUSTA = "Jackie Chan"
CUSTB = "Jet Li"

# initialize customer hours map to zero
CustHours = {CUSTA: 0, CUSTB: 0}

for report in glob.glob("C:\*.xlsx"):
    print "Processing :"+report
    book = xlrd.open_workbook(report)
    sh=book.sheet_by_index(0)
    for rx in range(1,sh.nrows):
        # if date in current week then add value to corresponding
        section

```

```

if sh.cell(rx,DATECOL).value <> "":
try:
date_value = xldate_as_tuple(sh.cell(rx,DATECOL).value,book.
datemode)
except:
print "error with date"
continue
t = date.today() - date("date_value[:3]")
if t.days <= ONeweek and sh.cell(rx,HOURSCOL).value <> "":
hoursspent = float(sh.cell(rx,HOURSCOL).value)
customer=str(sh.cell(rx,CUSTOMERCOL).value)
# summarize per customer also
# get project hours if already present
CustHours[customer]+=hoursspent

# print the summaries
print "Summary per customer"
print CUSTA+" "+str(CustHours[CUSTA])+" hours"
print CUSTB+" "+str(CustHours[CUSTB])+" hours"

```



Note 1: Note that we print the report being processed to help in debugging, in case any data makes the program misbehave.

Note 2: We process `xldate_as_tuple` in a `try` block so that we handle any exceptions thrown by incorrect dates. We print an error message in that case, which along with the report file name will help us know where the problem is.

When run through Python, the above program produces the following output:

```

Processing :C:\test.xlsx
Summary per customer
Jackie Chan:24.0 hours
Jet Li:16.0 hours

```

More ideas

Think about how you can improve the above solution. To start with, we could make all those constants configurable and pass them as input to our program through a `cfg` file (refer to 'Practical Python Programming' in the February 2015 issue of *OSFY* for how to use a `ConfigParser` module).

We could use better formatting to output the summary (refer `repr()` and `format()` interfaces).

Also, readers familiar with pivot tables could start thinking about how to implement generating such pivots from Python. 

By: T V Krishnamurthy

The author has more than 15 years of telecom industry experience. He has been actively involved in radio technologies for nearly a decade while working with Siemens Communications, Nokia Siemens Networks and Radisys Corporation. He lives and works at Bengaluru.